



Université Cheikh Anta Diop de
Dakar

Faculté des Sciences et Techniques
Département de Mathématiques et Informatique



Laboratoire d'Algèbre de Cryptologie
LAGCAA

Master Transmission de Données et Sécurité de l'Information

Option : Cryptographie-Informatique

Sujet :

Analyse comparative des modes
d'opération d'AES
(CBC vs GCM)
Sécurité face aux attaques actives
et performances dans la
transmission de données

Projet présenté par

Koffi Cyril SEPLEDJI

Sous la direction de

Dr. Babacar Alassane NDAW

Le 17 Mai 2026

Année académique 2025–2026

Table des matières

1	Introduction	4
2	Généralités sur la cryptographie symétrique	4
2.1	Principe de la cryptographie symétrique	4
2.2	Chiffrement et déchiffrement	5
2.3	Gestion et distribution des clés secrètes	5
2.4	Modes d'opération des chiffrements par blocs	6
3	L'algorithme AES	7
3.1	Présentation générale d'AES	7
3.2	Structure et fonctionnement d'AES	7
3.2.1	Représentation des octets dans le corps fini $GF(2^8)$	7
3.2.2	Représentation des mots de 32 bits	8
3.3	Les transformations d'AES	8
4	L'algorithme AES	8
4.1	Les transformations d'AES	8
4.1.1	La transformation SubBytes	9
4.1.2	La transformation ShiftRows	11
4.1.3	La transformation MixColumns	12
4.1.4	La transformation AddRoundKey	15
4.1.5	Enchaînement des quatre transformations	16
5	Le mode d'opération CBC	17
5.1	Principe de fonctionnement	17
5.2	Chiffrement et déchiffrement	20
5.3	Forces et limites du mode CBC	23
5.4	Vulnérabilités face aux attaques actives	25
6	Mode d'opération GCM	26
6.1	Chiffrement et authentification	26
6.1.1	Principe général	26
6.1.2	Construction du compteur initial	27
6.1.3	Chiffrement du message	27
6.1.4	Données associées (AAD)	27
6.1.5	Fonction GHASH	28
6.1.6	Génération du tag d'authentification	28
6.1.7	Procédure complète de chiffrement	28
6.2	Gestion du nonce	29
6.2.1	Pourquoi la réutilisation du nonce est-elle dangereuse?	29
6.2.2	Bonnes pratiques de gestion	29
6.3	Forces et limites du mode GCM	30
6.3.1	Forces du mode GCM	30
6.3.2	Limites du mode GCM	30
6.3.3	Synthèse des propriétés	31
6.4	Résistance aux attaques actives	31
6.4.1	Modification du texte chiffré	31
6.4.2	Injection d'un faux message	32
6.4.3	Attaque par modification des AAD	32
6.4.4	Attaque par replay	32

6.4.5	Bilan de la résistance aux attaques actives	33
7	Analyse comparative de CBC et GCM	33
7.1	Comparaison du niveau de sécurité	33
7.2	Comparaison face aux attaques actives	33
7.3	Comparaison des performances	33
7.4	Avantages et limites de chaque mode	33
8	Expérimentations et résultats	33
8.1	Environnement expérimental	33
8.2	Scénarios de test	33
8.3	Mesure des performances	33
8.4	Analyse des résultats	33
9	Conclusion et perspectives	33

1 Introduction

La sécurisation des données échangées dans les systèmes numériques repose largement sur l'utilisation de mécanismes cryptographiques capables d'assurer leur confidentialité et leur intégrité. Parmi les algorithmes de chiffrement symétrique les plus utilisés, l'*Advanced Encryption Standard* (AES) occupe une place importante grâce à son niveau de sécurité, son efficacité et ses nombreuses applications dans les systèmes informatiques et les réseaux.

Cependant, la sécurité d'une communication ne dépend pas uniquement de l'algorithme AES, mais également du mode d'opération utilisé et de la manière dont celui-ci est mis en œuvre. Cette étude s'intéresse particulièrement aux modes CBC (*Cipher Block Chaining*) et GCM (*Galois/Counter Mode*), qui présentent des mécanismes et des propriétés de sécurité différents. Alors que CBC assure principalement le chiffrement des données et nécessite un mécanisme complémentaire pour garantir leur intégrité, GCM associe directement chiffrement et authentification dans un même mécanisme.

Cette différence soulève également des questions concernant la résistance de ces deux modes face aux attaques actives, ainsi que leur influence sur les performances d'une communication sécurisée. Le choix d'un mode d'opération doit en effet prendre en compte à la fois les exigences de sécurité et les contraintes liées au temps de traitement et au volume de données à transmettre.

La problématique étudiée est donc la suivante :

Quel est l'impact du choix entre les modes CBC et GCM sur la sécurité d'une communication face aux attaques actives, et quelles différences de performances peut-on observer lors de la transmission des données ?

L'objectif de ce travail est de comparer ces deux modes à travers une étude théorique et expérimentale portant sur leur fonctionnement, leur sécurité face aux attaques actives et leurs performances. Une attention particulière sera accordée aux mécanismes d'authentification, aux principales vulnérabilités associées à chaque mode ainsi qu'au temps nécessaire au traitement des données. Les résultats permettront ainsi de mettre en évidence les avantages et les limites de chacun des modes étudiés et de mieux comprendre les critères à prendre en compte lors du choix d'un mode d'opération d'AES.

2 Généralités sur la cryptographie symétrique

2.1 Principe de la cryptographie symétrique

La cryptographie symétrique est une technique de chiffrement dans laquelle une même clé secrète est utilisée pour chiffrer et déchiffrer les données. L'émetteur transforme ainsi le message original, appelé texte clair, en un message chiffré qui ne peut être compris sans disposer de la clé appropriée.

Le principe général peut être représenté par la relation suivante :

$$C = E_K(M) \quad (1)$$

où M représente le message en clair, K la clé secrète, E la fonction de chiffrement et C le message chiffré.

Le processus inverse permet de retrouver le message original :

$$M = D_K(C) \quad (2)$$

où D représente la fonction de déchiffrement.

La sécurité d'un système de chiffrement symétrique repose principalement sur la confidentialité de la clé. En effet, une personne disposant de cette clé peut généralement déchiffrer les données

protégées. La gestion et la distribution des clés constituent donc des éléments essentiels dans la conception d'un système cryptographique.

Les algorithmes symétriques sont particulièrement adaptés au traitement de grandes quantités de données en raison de leur efficacité et de leur rapidité. Ils sont ainsi largement utilisés dans les systèmes informatiques, les réseaux et les protocoles de communication sécurisés.

2.2 Chiffrement et déchiffrement

Dans un système de cryptographie symétrique, le chiffrement et le déchiffrement constituent deux opérations complémentaires reposant sur une même clé secrète. Le chiffrement permet de transformer un message en clair en une représentation chiffrée, tandis que le déchiffrement permet de reconstruire le message original à partir du texte chiffré.

Soit M un message en clair et K une clé secrète. Le chiffrement de M à l'aide de la clé K peut être représenté par :

$$C = E_K(M) \quad (3)$$

où E_K désigne la fonction de chiffrement associée à la clé K et C le texte chiffré obtenu.

Le déchiffrement consiste alors à appliquer la fonction inverse afin de retrouver le message original :

$$M = D_K(C) \quad (4)$$

où D_K représente la fonction de déchiffrement.

Pour qu'un système cryptographique soit correctement fonctionnel, les opérations de chiffrement et de déchiffrement doivent satisfaire la propriété :

$$D_K(E_K(M)) = M \quad (5)$$

Cette relation signifie que le déchiffrement d'un message précédemment chiffré avec la même clé permet de retrouver exactement le message initial.

Dans la pratique, la sécurité d'un algorithme symétrique ne doit pas reposer sur le secret de l'algorithme lui-même, mais principalement sur la protection de la clé cryptographique. Une clé compromise peut en effet permettre à un adversaire disposant du texte chiffré de retrouver les données protégées.

La cryptographie symétrique présente également l'avantage d'être relativement rapide et adaptée au chiffrement de volumes importants de données. C'est notamment pour cette raison que des algorithmes tels que l'AES sont largement utilisés dans les systèmes de communication et de stockage sécurisés.

Cependant, lorsqu'un chiffrement par blocs est utilisé pour protéger des données dont la longueur dépasse celle d'un bloc, une question supplémentaire apparaît : *comment appliquer correctement l'algorithme de chiffrement à l'ensemble du message ?*

Cette problématique conduit à l'utilisation des **modes d'opération des chiffrements par blocs**, qui définissent la manière dont les différents blocs de données sont traités et combinés.

2.3 Gestion et distribution des clés secrètes

La sécurité d'un système de chiffrement symétrique dépend fortement de la protection de la clé secrète. En effet, contrairement au message chiffré qui peut être transmis sur un canal non sécurisé, la clé doit rester confidentielle et ne doit être accessible qu'aux entités autorisées.

Dans une communication entre deux parties, une première difficulté consiste donc à établir une clé secrète commune avant l'échange des données. Cette opération peut devenir complexe

lorsque le nombre d'utilisateurs augmente. Si chaque paire d'utilisateurs doit disposer d'une clé distincte, le nombre de clés nécessaires pour n utilisateurs est donné par :

$$N = \frac{n(n-1)}{2} \quad (6)$$

Cette croissance quadratique rend la gestion des clés de plus en plus difficile dans les systèmes comportant un grand nombre d'utilisateurs. À cela s'ajoutent les problèmes liés au stockage sécurisé, au renouvellement périodique des clés, à leur distribution et à leur révocation lorsqu'elles sont compromises.

La distribution des clés constitue ainsi l'une des principales limites pratiques de la cryptographie symétrique. Plusieurs mécanismes peuvent être utilisés pour résoudre ce problème, notamment l'utilisation d'un serveur de gestion de clés ou le recours à des mécanismes de cryptographie asymétrique permettant d'établir une clé secrète de manière sécurisée.

Une fois la clé établie, le système symétrique présente toutefois un avantage important : les opérations de chiffrement et de déchiffrement sont généralement rapides et peuvent être appliquées efficacement à de grandes quantités de données.

La gestion de la clé ne constitue cependant pas la seule considération. Pour les chiffrements par blocs, il est également nécessaire de définir la manière dont les blocs successifs du message sont traités. Cette fonction est assurée par les modes d'opération, dont dépendent notamment certaines propriétés de sécurité et les performances du chiffrement.

2.4 Modes d'opération des chiffrements par blocs

Un chiffrement par blocs traite les données sous forme de blocs de taille fixe. Lorsqu'un message est plus long qu'un seul bloc, il est nécessaire de définir une méthode permettant d'appliquer l'algorithme de chiffrement à l'ensemble des données. Cette méthode est appelée *mode d'opération*.

Un mode d'opération définit notamment la manière dont les blocs de données successifs sont traités et, selon le mode considéré, comment ils sont liés les uns aux autres. Il peut également introduire des paramètres supplémentaires tels qu'un vecteur d'initialisation (*Initialization Vector*, IV) ou un nonce.

Le choix du mode d'opération ne constitue pas un simple choix d'implémentation. Il influence directement les propriétés de sécurité du système ainsi que son comportement lors du traitement des données. Certains modes assurent principalement la confidentialité, tandis que d'autres permettent également de garantir l'intégrité et l'authenticité des données.

Parmi les différents modes d'opération utilisés avec les chiffrements par blocs, on peut notamment citer ECB (*Electronic Codebook*), CBC (*Cipher Block Chaining*), CFB (*Cipher Feedback*), OFB (*Output Feedback*) et CTR (*Counter Mode*). Ces modes présentent des mécanismes de fonctionnement et des propriétés de sécurité différentes.

Dans le cadre de cette étude, l'intérêt porte particulièrement sur deux modes associés à l'algorithme AES : CBC et GCM. Le mode CBC repose sur un mécanisme de chaînage entre les blocs et assure principalement la confidentialité des données. Le mode GCM, quant à lui, combine le chiffrement avec un mécanisme d'authentification permettant également de détecter certaines modifications des données.

L'étude de ces modes nécessite donc de considérer à la fois leur fonctionnement interne, leurs exigences de sécurité et leur influence sur les performances. Ces éléments constituent la base de la comparaison qui sera développée dans les sections consacrées à CBC et GCM.

3 L'algorithme AES

3.1 Présentation générale d'AES

L'*Advanced Encryption Standard* (AES) est un standard de chiffrement symétrique par blocs destiné notamment à remplacer le *Data Encryption Standard* (DES). Le développement de l'AES a été initié par le *National Institute of Standards and Technology* (NIST) en janvier 1997, avant son adoption comme standard en novembre 2001.

L'AES est fondé sur l'algorithme *Rijndael*, conçu par les cryptographes Joan Daemen et Vincent Rijmen. Son développement s'est appuyé sur un processus ouvert de consultation et d'analyse auquel ont participé des experts internationaux.

Sur le plan structurel, AES est un chiffrement par blocs itératif. Contrairement au DES, il ne repose pas sur un réseau de Feistel. Sa construction s'appuie principalement sur une succession de substitutions et de transpositions, complétées par des opérations algébriques sur les éléments manipulés par l'algorithme.

L'algorithme original *Rijndael* permet de considérer plusieurs combinaisons de longueurs de blocs et de clés, notamment 128, 192 et 256 bits. Le nombre de tours de chiffrement dépend de la configuration retenue et peut être de 10, 12 ou 14 tours. Cette combinaison des longueurs de bloc et de clé conduit, pour Rijndael, à plusieurs configurations possibles.

Dans le cas de l'AES standardisé par le NIST, la taille du bloc est fixée à 128 bits. Les trois longueurs de clé retenues sont 128, 192 et 256 bits, associées respectivement à 10, 12 et 14 tours de transformation.

Ainsi, la structure d'AES repose sur l'application répétée de transformations cryptographiques destinées à transformer l'état interne du bloc au cours des différentes rondes. L'étude de ces transformations permet de mieux comprendre le fonctionnement interne de l'algorithme et prépare l'analyse des modes d'opération CBC et GCM étudiés dans la suite de ce travail.

3.2 Structure et fonctionnement d'AES

L'algorithme AES opère sur des blocs de données en clair de 128 bits qu'il transforme en blocs chiffrés de 128 bits à partir d'une clé secrète de 128, 192 ou 256 bits. Le nombre de tours (*rounds*) effectués au cours du chiffrement dépend de la longueur de la clé : 10 tours pour une clé de 128 bits, 12 tours pour une clé de 192 bits et 14 tours pour une clé de 256 bits.

Le traitement interne d'AES repose sur des opérations effectuées sur des octets et sur des mots de 32 bits. Ces opérations sont définies à partir de structures algébriques permettant de réaliser les différentes transformations du chiffrement.

3.2.1 Représentation des octets dans le corps fini $\text{GF}(2^8)$

Un octet est constitué de huit bits. Il peut être représenté par un polynôme de degré inférieur ou égal à 7 à coefficients dans $\text{GF}(2)$:

$$b(x) = b_7x^7 + b_6x^6 + \cdots + b_1x + b_0 \quad (7)$$

où chaque coefficient b_i appartient à $\{0, 1\}$.

Dans AES, les opérations sur les octets sont réalisées dans le corps fini $\text{GF}(2^8)$. L'addition de deux éléments correspond à l'addition des coefficients modulo 2 et est donc équivalente à une opération XOR bit à bit :

$$a(x) \oplus b(x) \quad (8)$$

La multiplication est effectuée comme une multiplication polynomiale classique, suivie d'une réduction modulo un polynôme irréductible de degré 8. Dans AES, le polynôme utilisé est :

$$m(x) = x^8 + x^4 + x^3 + x + 1 \quad (9)$$

Ainsi, la multiplication de deux éléments $a(x)$ et $b(x)$ est réalisée selon :

$$c(x) = a(x)b(x) \bmod m(x) \quad (10)$$

Cette construction permet de disposer de $2^8 = 256$ éléments et fournit le cadre algébrique utilisé par plusieurs transformations internes d'AES.

3.2.2 Représentation des mots de 32 bits

AES manipule également des mots de 32 bits, chacun étant constitué de quatre octets. Un tel mot peut être représenté comme un polynôme de degré inférieur ou égal à 3 dont les coefficients appartiennent au corps $\text{GF}(2^8)$:

$$B(x) = b_3x^3 + b_2x^2 + b_1x + b_0 \quad (11)$$

Les opérations sur ces mots sont alors définies dans l'anneau de polynômes $\text{GF}(2^8)[x]$. Lorsqu'une multiplication produit un polynôme de degré supérieur à 3, celui-ci est réduit modulo :

$$M(x) = x^4 + 1 \quad (12)$$

Cette représentation algébrique intervient notamment dans la transformation *MixColumns*, où les quatre octets d'une colonne de l'état sont traités comme un mot de 32 bits.

La compréhension de ces structures permet ainsi d'aborder les principales transformations qui constituent les différentes rondes du chiffrement AES : *SubBytes*, *ShiftRows*, *MixColumns* et *AddRoundKey*.

3.3 Les transformations d'AES

4 L'algorithme AES

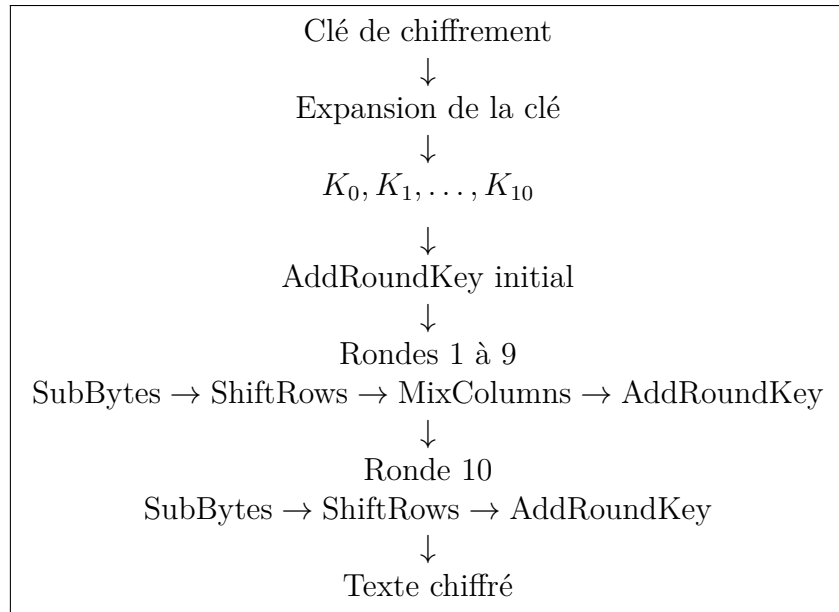
4.1 Les transformations d'AES

L'algorithme AES repose sur quatre transformations fondamentales : *SubBytes*, *ShiftRows*, *MixColumns* et *AddRoundKey*. Ces transformations sont appliquées au tableau d'état au cours des différentes rondes du chiffrement.

Pour AES, la taille du bloc est fixée à 128 bits, soit 16 octets. Le tableau d'état est donc constitué de quatre lignes et de quatre colonnes. Chaque élément du tableau représente un octet appartenant au corps fini $\text{GF}(2^8)$.

$$State = \begin{bmatrix} s_{0,0} & s_{0,1} & s_{0,2} & s_{0,3} \\ s_{1,0} & s_{1,1} & s_{1,2} & s_{1,3} \\ s_{2,0} & s_{2,1} & s_{2,2} & s_{2,3} \\ s_{3,0} & s_{3,1} & s_{3,2} & s_{3,3} \end{bmatrix}$$

Pour AES-128, le chiffrement comporte dix rondes. Une transformation *AddRoundKey* est d'abord appliquée avant la première ronde. Les rondes 1 à 9 utilisent les quatre transformations, tandis que la dixième ronde ne comporte pas *MixColumns*.



4.1.1 La transformation SubBytes

La transformation *SubBytes*, appelée également *ByteSub*, consiste à substituer indépendamment chacun des octets du tableau d'état par son image dans une table de substitution appelée S-Box.

Si $s_{i,j}$ désigne un octet de l'état, alors :

$$s'_{i,j} = S(s_{i,j})$$

où S représente la S-Box d'AES.

La transformation est appliquée indépendamment à chacun des 16 octets de l'état.

Construction mathématique de la S-Box. La S-Box AES est construite à partir des propriétés du corps fini $\text{GF}(2^8)$. Pour un élément non nul a , on détermine d'abord son inverse multiplicatif :

$$a^{-1} \in \text{GF}(2^8).$$

Une transformation affine est ensuite appliquée à cet inverse.

Le principe peut être représenté par :

$$a \longrightarrow a^{-1} \longrightarrow \text{transformation affine} \longrightarrow S(a)$$

avec la convention :

$$S(0) = 0.$$

La S-Box ainsi obtenue est bijective. Son inverse peut donc être utilisé lors du déchiffrement :

$$S^{-1}(S(a)) = a$$

Table de substitution S-Box. La S-Box est organisée sous la forme d'un tableau 16×16 . Pour un octet représenté sous la forme hexadécimale xy , le premier chiffre hexadécimal détermine la ligne et le second détermine la colonne.

	TABLE 1 – S-Box AES															
	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	63	7c	77	7b	f2	6b	6f	c5	30	01	67	2b	fe	d7	ab	76
1	ca	82	c9	7d	fa	59	47	f0	ad	d4	a2	af	9c	a4	72	c0
2	b7	fd	93	26	36	3f	f7	cc	34	a5	e5	f1	71	d8	31	15
3	04	c7	23	c3	18	96	05	9a	07	12	80	e2	eb	27	b2	75
4	09	83	2c	1a	1b	6e	5a	a0	52	3b	d6	b3	29	e3	2f	84
5	53	d1	00	ed	20	fc	b1	5b	6a	cb	be	39	4a	4c	58	cf
6	d0	ef	aa	fb	43	4d	33	85	45	f9	02	7f	50	3c	9f	a8
7	51	a3	40	8f	92	9d	38	f5	bc	b6	da	21	10	ff	f3	d2
8	cd	0c	13	ec	5f	97	44	17	c4	a7	7e	3d	64	5d	19	73
9	60	81	4f	dc	22	2a	90	88	46	ee	b8	14	de	5e	0b	db
A	e0	32	3a	0a	49	06	24	5c	c2	d3	ac	62	91	95	e4	79
B	e7	c8	37	6d	8d	d5	4e	a9	6c	56	f4	ea	65	7a	ae	08
C	ba	78	25	2e	1c	a6	b4	c6	e8	dd	74	1f	4b	bd	8b	8a
D	70	3e	b5	66	48	03	f6	0e	61	35	57	b9	86	c1	1d	9e
E	e1	f8	98	11	69	d9	8e	94	9b	1e	87	e9	ce	55	28	df
F	8c	a1	89	0d	bf	e6	42	68	41	99	2d	0f	b0	54	bb	16

Exemple de substitution d'un octet. Considérons l'octet :

$$a = 53.$$

Le premier chiffre hexadécimal est 5 et le second est 3. Nous sélectionnons donc la ligne 5 et la colonne 3 de la S-Box.

On obtient :

$$S(53) = \text{ED}$$

Ainsi :

$$53 \xrightarrow{\text{S-Box}} \text{ED}$$

Un autre exemple est :

$$S(19) = \text{D4}.$$

Application à un tableau d'état. Considérons le tableau :

$$\text{State} = \begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix}.$$

Chaque octet est remplacé indépendamment.

Par exemple :

$$S(19) = D4, \quad S(A0) = E0,$$

$$S(9A) = B8, \quad S(E9) = 1E.$$

Après application de la S-Box à tous les octets :

$$\begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix} \xrightarrow{\text{SubBytes}} \begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}$$

4.1.2 La transformation ShiftRows

La transformation *ShiftRows* réalise un décalage cyclique vers la gauche des lignes du tableau d'état.

Pour AES, les décalages sont :

TABLE 2 – Décalage des lignes lors de ShiftRows

Ligne	Décalage cyclique
0	0
1	1
2	2
3	3

Pour un état général :

$$State = \begin{bmatrix} a_0 & a_1 & a_2 & a_3 \\ b_0 & b_1 & b_2 & b_3 \\ c_0 & c_1 & c_2 & c_3 \\ d_0 & d_1 & d_2 & d_3 \end{bmatrix}$$

on obtient après *ShiftRows* :

$$State' = \begin{bmatrix} a_0 & a_1 & a_2 & a_3 \\ b_1 & b_2 & b_3 & b_0 \\ c_2 & c_3 & c_0 & c_1 \\ d_3 & d_0 & d_1 & d_2 \end{bmatrix}$$

Pour AES-128, on peut également écrire :

$$s'_{i,j} = s_{i,(j+i) \bmod 4}$$

où i représente le numéro de ligne et j le numéro de colonne.

Exemple numérique. Partons de l'état obtenu après *SubBytes* :

$$State = \begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix}.$$

Les quatre lignes sont transformées comme suit :

$$(d4, e0, b8, 1e) \longrightarrow (d4, e0, b8, 1e)$$

$$(27, bf, b4, 41) \longrightarrow (bf, b4, 41, 27)$$

$$(11, 98, 5d, 52) \longrightarrow (5d, 52, 11, 98)$$

$$(ae, f1, e5, 30) \longrightarrow (30, ae, f1, e5)$$

On obtient finalement :

$$\boxed{\begin{bmatrix} d4 & e0 & b8 & 1e \\ 27 & bf & b4 & 41 \\ 11 & 98 & 5d & 52 \\ ae & f1 & e5 & 30 \end{bmatrix} \xrightarrow{\text{ShiftRows}} \begin{bmatrix} d4 & e0 & b8 & 1e \\ bf & b4 & 41 & 27 \\ 5d & 52 & 11 & 98 \\ 30 & ae & f1 & e5 \end{bmatrix}}$$

Cette transformation ne modifie donc pas la valeur des octets. Elle modifie uniquement leur position.

4.1.3 La transformation MixColumns

La transformation *MixColumns* réalise un mélange des octets au niveau de chaque colonne du tableau d'état.

Une colonne est considérée comme un vecteur de quatre éléments de $\text{GF}(2^8)$:

$$C = \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}.$$

AES utilise la matrice constante :

$$M = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix}.$$

Le calcul est effectué dans $\text{GF}(2^8)$:

$$\boxed{C' = M \times C}$$

soit :

$$\begin{bmatrix} a' \\ b' \\ c' \\ d' \end{bmatrix} = \begin{bmatrix} 02 & 03 & 01 & 01 \\ 01 & 02 & 03 & 01 \\ 01 & 01 & 02 & 03 \\ 03 & 01 & 01 & 02 \end{bmatrix} \begin{bmatrix} a \\ b \\ c \\ d \end{bmatrix}.$$

On obtient :

$$a' = (02 \otimes a) \oplus (03 \otimes b) \oplus c \oplus d$$

$$b' = a \oplus (02 \otimes b) \oplus (03 \otimes c) \oplus d$$

$$c' = a \oplus b \oplus (02 \otimes c) \oplus (03 \otimes d)$$

$$d' = (03 \otimes a) \oplus b \oplus c \oplus (02 \otimes d).$$

Le symbole $\oplus$ représente l'addition dans $\text{GF}(2^8)$, correspondant à un XOR bit à bit, tandis que $\otimes$ représente la multiplication dans ce corps.

Calcul dans $\text{GF}(2^8)$. Les opérations de *MixColumns* sont réalisées dans le corps fini $\text{GF}(2^8)$ à l'aide du polynôme irréductible :

$$m(x) = x^8 + x^4 + x^3 + x + 1$$

Un octet est représenté par un polynôme à coefficients dans $\text{GF}(2)$ et de degré inférieur ou égal à 7.

Par exemple :

$$57 = 00111001_2$$

correspond au polynôme :

$$x^5 + x^4 + x^3 + 1.$$

La multiplication dans $\text{GF}(2^8)$ est effectuée par multiplication polynomiale suivie d'une réduction modulo $m(x)$.

Exemple : multiplication par 02. Considérons :

$$a = 57.$$

En binaire :

$$57 = 00111001_2.$$

Le bit de poids fort étant nul, on effectue un décalage d'un bit vers la gauche :

$$01010111_2 \longrightarrow 10101110_2.$$

On obtient :

$$02 \otimes 57 = \text{AE}$$

Exemple : multiplication par 03. Dans $\text{GF}(2^8)$:

$$03 \otimes a = (02 \otimes a) \oplus a.$$

Ainsi :

$$03 \otimes 57 = \text{AE} \oplus 57.$$

En binaire :

$$\begin{array}{r} 10101110 \\ \oplus 01010111 \\ \hline 11111001 \end{array}$$

donc :

$$\boxed{03 \otimes 57 = F9}$$

Exemple complet de MixColumns. Considérons la colonne :

$$C = \begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix}.$$

La première composante du résultat est :

$$a' = (02 \otimes d4) \oplus (03 \otimes bf) \oplus 5d \oplus 30.$$

Les opérations dans $GF(2^8)$ donnent :

$$02 \otimes d4 = B3$$

et :

$$02 \otimes bf = 65.$$

Par conséquent :

$$03 \otimes bf = (02 \otimes bf) \oplus bf$$

$$= 65 \oplus BF$$

$$= DA.$$

On obtient alors :

$$a' = B3 \oplus DA \oplus 5D \oplus 30.$$

Calculons successivement :

$$B3 \oplus DA = 69$$

$$69 \oplus 5D = 34$$

$$34 \oplus 30 = 04.$$

Donc :

$$\boxed{a' = 04}$$

En appliquant les mêmes opérations aux trois autres composantes :

$$\begin{bmatrix} d4 \\ bf \\ 5d \\ 30 \end{bmatrix} \xrightarrow{\text{MixColumns}} \begin{bmatrix} 04 \\ 66 \\ 81 \\ e5 \end{bmatrix}$$

Ainsi, chaque octet de la nouvelle colonne dépend des quatre octets de la colonne initiale.

Interprétation polynomiale. Une colonne :

$$(a_0, a_1, a_2, a_3)$$

peut être représentée par le polynôme :

$$a(x) = a_0x^3 + a_1x^2 + a_2x + a_3.$$

Le polynôme constant utilisé par AES est :

$$c(x) = 03x^3 \oplus 01x^2 \oplus 01x \oplus 02.$$

La transformation peut alors s'écrire :

$$b(x) = c(x)a(x) \pmod{x^4 + 1}$$

où les coefficients appartiennent à $\text{GF}(2^8)$.

Cette formulation polynomiale est équivalente à la formulation matricielle présentée précédemment.

4.1.4 La transformation **AddRoundKey**

La transformation *AddRoundKey* consiste à combiner le tableau d'état avec la sous-clé correspondant au round considéré.

L'opération utilisée est le XOR :

$$State' = State \oplus K_i$$

où K_i représente la sous-clé du round i .

Cette opération est réalisée élément par élément entre le tableau d'état et la sous-clé correspondante.

Exemple sur un octet. Considérons :

$$State = 3A$$

et :

$$K_i = 5C.$$

En représentation binaire :

$$3A = 00111010$$

$$5C = 01011100.$$

Le XOR donne :

	0	0	1	1	1	0	1	0
$\oplus$	0	1	0	1	1	1	0	0
	0	1	1	0	0	1	1	0

Ainsi :

$$3A \oplus 5C = 66$$

Exemple matriciel. Supposons :

$$State = \begin{bmatrix} 19 & a0 & 9a & e9 \\ 3d & f4 & c6 & f8 \\ e3 & e2 & 8d & 48 \\ be & 2b & 2a & 08 \end{bmatrix}$$

et une sous-clé :

$$K_i = \begin{bmatrix} a0 & 88 & 23 & 2a \\ fa & 54 & a3 & 6c \\ fe & 2c & 39 & 76 \\ 17 & b1 & 39 & 05 \end{bmatrix}.$$

L'opération est effectuée élément par élément :

$$State' = State \oplus K_i.$$

Par exemple :

$$19 \oplus A0 = B9$$

et :

$$A0 \oplus 88 = 28.$$

Le résultat est donc obtenu en appliquant le même XOR à chacun des éléments correspondants.

4.1.5 Enchaînement des quatre transformations

Les quatre transformations précédentes interviennent de manière ordonnée au cours d'une ronde AES.

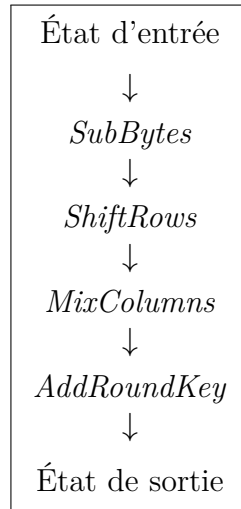
Pour une ronde standard, l'enchaînement est :

$$\boxed{State_i \xrightarrow{\text{SubBytes}} State'_i \xrightarrow{\text{ShiftRows}} State''_i \xrightarrow{\text{MixColumns}} State'''_i \xrightarrow{\text{AddRoundKey}} State_{i+1}}$$

On peut résumer le rôle de chaque transformation par le tableau suivant :

TABLE 3 – Rôle des principales transformations d'AES

Transformation	Nature	Fonction principale
<i>SubBytes</i>	Substitution non linéaire	Introduire la non-linéarité
<i>ShiftRows</i>	Permutation	Répartir les octets
<i>MixColumns</i>	Transformation linéaire	Assurer la diffusion
<i>AddRoundKey</i>	XOR avec la sous-clé	Introduire la clé



La dernière ronde d'AES constitue une exception importante : *MixColumns* n'y est pas appliquée. Ainsi, pour la dernière ronde, l'ordre des transformations est :

$$\boxed{\text{SubBytes} \rightarrow \text{ShiftRows} \rightarrow \text{AddRoundKey}}$$

L'ensemble de ces transformations permet à AES de combiner substitution non linéaire, permutation, diffusion et incorporation de la clé secrète. Elles constituent ainsi le mécanisme interne du chiffrement AES par blocs, avant l'étude des modes d'opération tels que CBC et GCM.

5 Le mode d'opération CBC

Le mode CBC (*Cipher Block Chaining*) est un mode d'opération utilisé avec les chiffrements symétriques par blocs, notamment AES. Son principe consiste à chaîner les blocs de données successifs afin que le chiffrement d'un bloc dépende également du bloc chiffré précédent.

Dans le cas d'AES, chaque bloc possède une longueur fixe de 128 bits, soit 16 octets. Le message en clair est donc découpé en blocs de 128 bits avant d'être traité par l'algorithme.

5.1 Principe de fonctionnement

Soit un message en clair constitué de n blocs :

$$P_1, P_2, \dots, P_n$$

où chaque bloc P_i possède une longueur de 128 bits.

Le mode CBC utilise également un vecteur d'initialisation, noté IV , de même longueur que le bloc AES :

$$\boxed{|IV| = 128 \text{ bits}}$$

Le vecteur d'initialisation est utilisé pour traiter le premier bloc. On peut alors considérer :

$$\boxed{C_0 = IV}$$

Le principe général du chiffrement CBC est représenté par :

$$\boxed{P_i \oplus C_{i-1} \longrightarrow E_K \longrightarrow C_i}$$

où :

- P_i représente le bloc de texte clair numéro i ;
- C_i représente le bloc chiffré numéro i ;
- C_{i-1} représente le bloc chiffré précédent ;
- $C_0 = IV$ pour le premier bloc ;
- K représente la clé secrète ;
- E_K représente la fonction de chiffrement AES avec la clé K ;
- $\oplus$ représente l'opération XOR.

La formule générale du chiffrement CBC est donc :

$$C_i = E_K(P_i \oplus C_{i-1})$$

avec :

$$C_0 = IV$$

Ainsi, le premier bloc est calculé selon :

$$C_1 = E_K(P_1 \oplus IV)$$

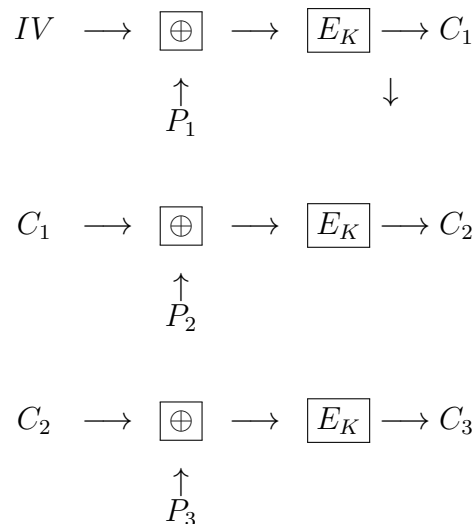
Le deuxième bloc dépend ensuite du premier bloc chiffré :

$$C_2 = E_K(P_2 \oplus C_1)$$

De manière générale :

$$C_i = E_K(P_i \oplus C_{i-1}) \quad \text{pour } 1 \leq i \leq n.$$

Représentation du chiffrement CBC. Le fonctionnement peut être représenté par le schéma suivant :



Ce schéma montre que chaque bloc de texte clair est d'abord combiné par XOR avec le bloc chiffré précédent avant d'être soumis au chiffrement AES.

Exemple numérique simplifié. Pour illustrer le principe du CBC, considérons trois blocs de données :

$$P_1, \quad P_2, \quad P_3$$

et un vecteur d'initialisation IV .

Le premier bloc est obtenu par :

$$P_1 \oplus IV$$

puis chiffré avec AES :

$$C_1 = E_K(P_1 \oplus IV)$$

Pour le deuxième bloc, le résultat précédent est utilisé :

$$P_2 \oplus C_1$$

puis :

$$C_2 = E_K(P_2 \oplus C_1)$$

Enfin, le troisième bloc est traité selon :

$$P_3 \oplus C_2$$

et :

$$C_3 = E_K(P_3 \oplus C_2)$$

L'ensemble du processus peut donc être résumé par :

$$\begin{aligned} C_1 &= E_K(P_1 \oplus IV), \\ C_2 &= E_K(P_2 \oplus C_1), \\ C_3 &= E_K(P_3 \oplus C_2). \end{aligned}$$

Illustration avec des valeurs hexadécimales. Considérons, à titre d'illustration du mécanisme XOR, les valeurs suivantes :

$$P_1 = 00112233445566778899AABBCCDDEEFF$$

et :

$$IV = 000102030405060708090A0B0C0D0E0F.$$

Avant le chiffrement AES, les deux valeurs sont combinées par XOR :

$$P_1 \oplus IV.$$

Le calcul est effectué octet par octet :

	00	11	22	33	44	55	66	77
<i>IV</i>	00	01	02	03	04	05	06	07
$\oplus$	00	10	20	30	40	50	60	70

et pour la seconde partie :

<i>P</i> ₁	88	99	AA	BB	CC	DD	EE	FF
<i>IV</i>	08	09	0A	0B	0C	0D	0E	0F
$\oplus$	80	90	A0	B0	C0	D0	E0	F0

Ainsi :

$$P_1 \oplus IV = 00102030405060708090A0B0C0D0E0F0$$

Ce résultat constitue l'entrée du chiffrement AES :

$$C_1 = E_K(00102030405060708090A0B0C0D0E0F0)$$

Le résultat exact de C_1 dépend naturellement de la clé K utilisée.

Principe du chaînage. Le terme *Cipher Block Chaining* signifie littéralement « chaînage des blocs chiffrés ». Cette appellation provient du fait que chaque nouveau bloc de texte clair est combiné avec le bloc chiffré précédent.

La dépendance peut être représentée par :

$$P_1 \rightarrow C_1 \rightarrow C_2 \rightarrow C_3$$

avec :

$$C_1 = E_K(P_1 \oplus IV),$$

$$C_2 = E_K(P_2 \oplus C_1),$$

$$C_3 = E_K(P_3 \oplus C_2).$$

Par conséquent, deux blocs de texte clair identiques ne produisent pas nécessairement le même bloc chiffré lorsqu'ils apparaissent à des positions différentes dans une même séquence CBC, puisque le bloc précédent intervient dans le calcul.

Rôle du vecteur d'initialisation. Le vecteur d'initialisation intervient uniquement au début de la chaîne :

$$C_0 = IV.$$

Il permet d'éviter que le premier bloc soit toujours chiffré à partir d'une même valeur initiale lorsque la même clé est utilisée.

Le IV n'a pas besoin d'être secret. Toutefois, pour le chiffrement CBC, son choix doit respecter les propriétés requises par le schéma d'utilisation, notamment son caractère imprévisible au moment de la génération.

Le fonctionnement du déchiffrement CBC repose sur le mécanisme inverse. Chaque bloc chiffré est d'abord soumis au déchiffrement AES, puis le résultat est combiné par XOR avec le bloc chiffré précédent.

La formule du déchiffrement sera étudiée dans la sous-section suivante.

5.2 Chiffrement et déchiffrement

Le principe général du chiffrement CBC a été présenté dans la section précédente. Pour le détail du mécanisme de chiffrement, des équations et de l'exemple numérique, se reporter à la section ??.

Le déchiffrement nécessite cependant une analyse particulière, car les opérations sont effectuées dans un ordre différent de celui du chiffrement.

Déchiffrement CBC

Lors du déchiffrement, chaque bloc chiffré C_i est d'abord traité par la fonction de déchiffrement AES D_K . Le résultat obtenu est ensuite combiné par une opération XOR avec le bloc chiffré précédent.

La relation générale du déchiffrement CBC est :

$$P_i = D_K(C_i) \oplus C_{i-1}$$

avec :

$$C_0 = IV$$

Ainsi, pour le premier bloc :

$$P_1 = D_K(C_1) \oplus IV$$

Pour le deuxième bloc :

$$P_2 = D_K(C_2) \oplus C_1$$

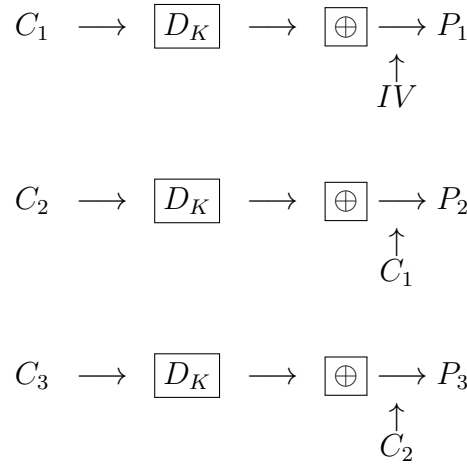
et pour le troisième :

$$P_3 = D_K(C_3) \oplus C_2$$

De manière générale :

$$P_i = D_K(C_i) \oplus C_{i-1}, \quad 1 \leq i \leq n.$$

Représentation du déchiffrement. Le fonctionnement du déchiffrement CBC peut être représenté comme suit :



Le schéma montre que le déchiffrement AES est effectué en premier, puis que le résultat est combiné avec le bloc précédent.

Justification mathématique. Le fonctionnement du déchiffrement peut être vérifié directement à partir de l'équation du chiffrement CBC :

$$C_i = E_K(P_i \oplus C_{i-1}).$$

En appliquant la fonction de déchiffrement D_K aux deux membres, on obtient :

$$D_K(C_i) = D_K(E_K(P_i \oplus C_{i-1})).$$

Puisque les fonctions E_K et D_K sont inverses :

$$D_K(E_K(x)) = x.$$

On obtient alors :

$$D_K(C_i) = P_i \oplus C_{i-1}.$$

Il suffit ensuite d'effectuer un XOR avec C_{i-1} :

$$D_K(C_i) \oplus C_{i-1} = (P_i \oplus C_{i-1}) \oplus C_{i-1}.$$

Or, le XOR possède les propriétés :

$$x \oplus x = 0$$

et :

$$x \oplus 0 = x.$$

Par conséquent :

$$\begin{aligned} D_K(C_i) \oplus C_{i-1} &= P_i \oplus C_{i-1} \oplus C_{i-1} \\ &= P_i \oplus 0 \\ &= P_i. \end{aligned}$$

Ainsi, on retrouve bien le bloc de texte clair initial :

$$\boxed{P_i = D_K(C_i) \oplus C_{i-1}.}$$

Exemple numérique. Considérons un bloc chiffré C_1 et un vecteur d'initialisation IV .

Après l'application du déchiffrement AES, supposons que l'on obtienne :

$$D_K(C_1) = 00102030405060708090A0B0C0D0E0F0$$

et que :

$$IV = 000102030405060708090A0B0C0D0E0F.$$

Le bloc de texte clair est alors obtenu par :

$$P_1 = D_K(C_1) \oplus IV.$$

Le calcul est effectué octet par octet :

$D_K(C_1)$	00	10	20	30	40	50	60	70
IV	00	01	02	03	04	05	06	07
P_1	00	11	22	33	44	55	66	77

Pour les huit octets suivants :

$D_K(C_1)$	80	90	A0	B0	C0	D0	E0	F0
IV	08	09	0A	0B	0C	0D	0E	0F
P_1	88	99	AA	BB	CC	DD	EE	FF

On retrouve donc :

$$\boxed{P_1 = 00112233445566778899AABBCCDDEEFF}$$

Cet exemple montre que le XOR avec le vecteur d'initialisation permet de retrouver le premier bloc de texte clair après le déchiffrement AES.

Cas des blocs suivants. Pour les blocs suivants, le vecteur d'initialisation n'est plus utilisé. Il est remplacé par le bloc chiffré précédent.

Ainsi :

$$P_2 = D_K(C_2) \oplus C_1$$

puis :

$$P_3 = D_K(C_3) \oplus C_2$$

et ainsi de suite jusqu'au dernier bloc :

$$P_n = D_K(C_n) \oplus C_{n-1}.$$

Le mécanisme complet peut donc être résumé par :

$$\begin{array}{l} P_1 = D_K(C_1) \oplus IV, \\ P_2 = D_K(C_2) \oplus C_1, \\ P_3 = D_K(C_3) \oplus C_2, \\ \vdots \\ P_n = D_K(C_n) \oplus C_{n-1}. \end{array}$$

Le déchiffrement CBC repose ainsi sur deux opérations successives : le déchiffrement du bloc par AES, puis le XOR avec le bloc chiffré précédent.

5.3 Forces et limites du mode CBC

Le mode CBC présente plusieurs propriétés intéressantes pour le chiffrement des données. Cependant, son utilisation comporte également certaines contraintes, notamment en matière d'authentification et de traitement des erreurs.

Forces du mode CBC Chaînage des blocs.

Le chiffrement CBC repose sur la relation :

$$C_i = E_K(P_i \oplus C_{i-1}).$$

Chaque bloc de texte clair dépend ainsi du bloc chiffré précédent. Deux blocs de texte clair identiques ne produisent donc pas nécessairement le même bloc chiffré.

Vecteur d'initialisation.

Pour le premier bloc, le vecteur d'initialisation est utilisé comme valeur initiale du chaînage :

$$C_0 = IV$$

d'où :

$$C_1 = E_K(P_1 \oplus IV).$$

Le IV permet ainsi d'introduire une valeur initiale dans le processus de chiffrement.

Limites du mode CBC Absence d'authentification intégrée.

Le mode CBC assure la confidentialité des données, mais ne fournit pas à lui seul de mécanisme permettant de vérifier leur intégrité ou leur authenticité.

$$\boxed{\text{Confidentialité} \neq \text{Authentification}}$$

Un mécanisme d'authentification complémentaire est donc nécessaire lorsque la détection des modifications est requise.

Nécessité du padding.

AES utilise des blocs de 128 bits. Lorsque la longueur du message n'est pas un multiple de cette taille, un mécanisme de remplissage (*padding*) doit être utilisé pour compléter le dernier bloc.

Propagation des erreurs.

Lors du déchiffrement, on a :

$$P_i = D_K(C_i) \oplus C_{i-1}.$$

Une modification de C_i peut donc rendre P_i incorrect et affecter également le calcul de P_{i+1} :

$$P_{i+1} = D_K(C_{i+1}) \oplus C_i.$$

Ainsi :

$$\boxed{C_i \text{ modifié} \Rightarrow P_i \text{ corrompu et } P_{i+1} \text{ affecté}}$$

Dépendance séquentielle.

Le chiffrement d'un bloc dépend du bloc chiffré précédent :

$$C_1 \rightarrow C_2 \rightarrow C_3 \rightarrow \dots \rightarrow C_n.$$

Les blocs ne peuvent donc pas être chiffrés indépendamment, ce qui limite le parallélisme du chiffrement.

Synthèse Les principales caractéristiques du mode CBC sont résumées dans le tableau suivant.

TABLE 4 – Principales forces et limites du mode CBC

Aspect	Caractéristique
Chaînage	Chaque bloc dépend du bloc chiffré précédent.
IV	Le premier bloc utilise un vecteur d'initialisation.
Authentification	CBC ne fournit pas d'authentification intégrée.
Padding	Un remplissage peut être nécessaire pour compléter le dernier bloc.
Propagation des erreurs	Une modification d'un bloc chiffré affecte le bloc correspondant et peut affecter le suivant.
Parallélisation	Le chiffrement présente une dépendance séquentielle entre les blocs.

En résumé, CBC assure la confidentialité des données grâce au chaînage des blocs. Toutefois, l'absence d'authentification intégrée, la gestion du padding et la propagation des modifications constituent des limites importantes face à un adversaire actif.

5.4 Vulnérabilités face aux attaques actives

Contrairement à une attaque passive, dans laquelle l'adversaire se contente d'observer les communications, une attaque active consiste notamment à modifier, supprimer ou injecter des données dans le flux échangé.

Le mode CBC présente certaines propriétés qui peuvent être exploitées lorsqu'il est utilisé sans mécanisme d'authentification complémentaire.

Modification du ciphertext Lors du déchiffrement CBC, chaque bloc est obtenu selon :

$$P_i = D_K(C_i) \oplus C_{i-1}.$$

Si un adversaire modifie le bloc C_{i-1} avant son arrivée chez le destinataire, la valeur obtenue après déchiffrement devient :

$$P'_i = D_K(C_i) \oplus C'_{i-1}.$$

Ainsi, une modification du ciphertext peut provoquer une modification contrôlée de certains bits du bloc de texte clair correspondant.

Cette propriété constitue une faiblesse importante lorsque CBC est utilisé sans authentification.

Attaque par oracle de padding Une autre vulnérabilité importante concerne les implémentations de CBC utilisant un mécanisme de padding.

Dans certaines conditions, un attaquant peut modifier progressivement le ciphertext et observer la réaction du système lors du déchiffrement. Si le système distingue indirectement un padding valide d'un padding invalide, ces informations peuvent être exploitées pour obtenir progressivement des informations sur le texte clair.

Le principe général peut être représenté ainsi :

Ciphertext modifié $\longrightarrow$ Déchiffrement $\longrightarrow$ Vérification du padding $\longrightarrow$ Réponse observable

La vulnérabilité ne provient donc pas d'une faiblesse du chiffrement AES lui-même, mais de l'utilisation de CBC avec un mécanisme de padding et d'une implémentation qui révèle une information exploitable à l'attaquant.

Manipulation du vecteur d'initialisation Le premier bloc est déchiffré selon :

$$P_1 = D_K(C_1) \oplus IV.$$

Une modification du IV entraîne donc une modification du premier bloc déchiffré :

$$P'_1 = D_K(C_1) \oplus IV'.$$

La différence entre les deux valeurs est :

$$P'_1 \oplus P_1 = IV' \oplus IV.$$

Cette propriété montre pourquoi le traitement du vecteur d'initialisation doit être réalisé avec précaution lorsqu'il est utilisé avec CBC.

Conséquence générale Les différentes vulnérabilités précédentes montrent que le chiffrement CBC seul ne permet pas de garantir l'intégrité et l'authenticité des données transmises.

TABLE 5 – Principales vulnérabilités du mode CBC

Vulnérabilité	Conséquence
Modification du ciphertext	Possibilité de provoquer des modifications du texte clair lors du déchiffrement.
Padding oracle	Exploitation des réponses liées à la validation du padding pour obtenir progressivement des informations sur le texte clair.
Manipulation du IV	Possibilité de modifier le premier bloc déchiffré lorsque le IV peut être contrôlé ou modifié.
Absence d'authentification	Le récepteur ne dispose pas, avec CBC seul, d'une garantie cryptographique d'intégrité et d'authenticité du ciphertext.

Ainsi, l'utilisation de CBC dans un environnement soumis à des attaques actives nécessite l'ajout d'un mécanisme d'authentification approprié. Cette limitation constitue l'un des principaux éléments de comparaison avec le mode GCM, qui associe chiffrement et authentification dans une même construction cryptographique.

6 Mode d'opération GCM

Le *Galois Counter Mode* (GCM) est un mode d'opération cryptographique conçu pour fournir simultanément deux propriétés fondamentales de sécurité : la **confidentialité** des données et leur **authentification**. Il est normalisé par le National Institute of Standards and Technology (NIST) dans la publication SP 800-38D.

Contrairement à un mode de chiffrement classique tel que CBC, qui assure principalement la confidentialité, GCM associe le chiffrement CTR (*Counter Mode*) à une fonction d'authentification fondée sur une opération de hachage dans le corps fini $GF(2^{128})$, appelée *GHASH*.

Cette combinaison fait de GCM un mode de chiffrement authentifié, communément désigné par l'acronyme **AEAD** (*Authenticated Encryption with Associated Data*).

Le fonctionnement général de GCM repose sur trois éléments :

1. le chiffrement des données à l'aide du mode compteur ;
2. l'authentification du texte chiffré et des données associées ;
3. la génération d'une étiquette d'authentification (*authentication tag*).

Ainsi, le destinataire ne vérifie pas uniquement que le message peut être déchiffré : il vérifie également que celui-ci n'a pas été modifié pendant sa transmission.

6.1 Chiffrement et authentification

6.1.1 Principe général

GCM utilise généralement le chiffrement par blocs AES avec une taille de bloc de 128 bits. La clé secrète est notée K et le nonce (IV , *Initialization Vector*) est noté IV .

Une première valeur importante est calculée à partir du chiffrement AES du bloc nul :

$$H = E_K(0^{128}), \quad (13)$$

où E_K désigne le chiffrement AES avec la clé K .

La valeur H , appelée *hash subkey*, est ensuite utilisée par la fonction GHASH afin de calculer l'authentification des données.

Le fonctionnement de GCM peut donc être séparé conceptuellement en deux mécanismes :

- **CTR** pour assurer la confidentialité ;
- **GHASH** pour assurer l'authenticité et l'intégrité.

6.1.2 Construction du compteur initial

Lorsque le nonce possède une longueur de 96 bits, ce qui constitue le cas recommandé et le plus courant, le compteur initial est défini par :

$$J_0 = IV \parallel 0^{31} \parallel 1, \quad (14)$$

où $\parallel$ représente la concaténation.

Le compteur utilisé pour chiffrer les blocs suivants est obtenu par incrémentation de la partie basse du compteur :

$$CTR_i = \text{inc}_{32}(J_0) \quad (15)$$

et, de manière générale,

$$CTR_i = \text{inc}_{32}^i(J_0). \quad (16)$$

6.1.3 Chiffrement du message

Soit le message en clair :

$$P = P_1 \parallel P_2 \parallel \dots \parallel P_n. \quad (17)$$

Pour chaque bloc P_i , GCM calcule une valeur pseudo-aléatoire à partir du compteur :

$$S_i = E_K(CTR_i). \quad (18)$$

Le texte chiffré est ensuite obtenu par une opération XOR :

$$C_i = P_i \oplus S_i. \quad (19)$$

Ainsi, le chiffrement d'un bloc ne dépend pas directement du bloc précédent. Cette propriété permet notamment à GCM d'exploiter efficacement le parallélisme disponible dans les processeurs modernes.

6.1.4 Données associées (AAD)

GCM permet également de traiter des données qui ne doivent pas être chiffrées mais qui doivent néanmoins être authentifiées.

Ces données sont appelées **AAD** (*Additional Authenticated Data*).

On peut noter ces données A . Elles peuvent par exemple contenir :

- un identifiant de session ;
- un numéro de séquence ;
- des informations d'en-tête ;
- des métadonnées d'un protocole réseau.

Les données A ne sont donc pas chiffrées, mais leur modification entraîne l'échec de la vérification du tag.

6.1.5 Fonction GHASH

L'authentification repose sur la fonction GHASH, qui effectue des multiplications dans le corps fini :

$$\text{GF}(2^{128}). \quad (20)$$

Pour simplifier la présentation, soient :

- A : les données associées ;
- C : le texte chiffré ;
- H : la sous-clé d'authentification ;
- X : la valeur produite par GHASH.

La fonction GHASH peut être représentée sous la forme :

$$X = \text{GHASH}_H(A, C). \quad (21)$$

Dans le calcul réel, les données sont découpées en blocs de 128 bits et complétées si nécessaire. Une valeur finale contenant notamment les longueurs de A et de C est également intégrée au calcul.

Pour une séquence de blocs $X_1, \dots, X_m$, la construction peut être exprimée récursivement par :

$$Y_0 = 0^{128}, \quad (22)$$

$$Y_i = (Y_{i-1} \oplus X_i) \times H, \quad 1 \leq i \leq m. \quad (23)$$

La multiplication est effectuée dans $\text{GF}(2^{128})$.

6.1.6 Génération du tag d'authentification

Une fois le résultat GHASH calculé, GCM produit le tag d'authentification à partir de la valeur chiffrée du compteur initial.

Le tag complet peut être représenté par :

$$T = E_K(J_0) \oplus \text{GHASH}_H(A, C). \quad (24)$$

Dans la pratique, le tag peut être tronqué à une longueur t :

$$T_t = \text{MSB}_t(E_K(J_0) \oplus \text{GHASH}_H(A, C)), \quad (25)$$

où MSB_t désigne les t bits de poids fort.

Le résultat final de l'opération de chiffrement est donc constitué de :

$$\boxed{C \parallel T} \quad (26)$$

avec :

- C : le texte chiffré ;
- T : le tag d'authentification.

6.1.7 Procédure complète de chiffrement

Le fonctionnement d'Alice peut être résumé comme suit :

1. Alice choisit un nonce IV unique ;
2. elle calcule la sous-clé $H = E_K(0^{128})$;
3. elle construit le compteur initial J_0 ;

4. elle chiffre les blocs du message avec le mode CTR ;
5. elle applique GHASH aux données associées et au texte chiffré ;
6. elle calcule le tag d'authentification ;
7. elle transmet (IV, C, T) à Bob.

Le nonce n'est généralement pas considéré comme secret. Il doit cependant respecter une contrainte essentielle : **il ne doit jamais être réutilisé avec la même clé.**

6.2 Gestion du nonce

La gestion du nonce constitue l'une des exigences de sécurité les plus importantes de GCM.

Le nonce, souvent appelé IV (*Initialization Vector*), est une valeur qui permet de rendre chaque opération de chiffrement distincte.

Dans le cas recommandé d'un nonce de 96 bits :

$$IV \in \{0, 1\}^{96}. \quad (27)$$

Le nonce n'a pas besoin d'être secret et peut donc être transmis avec le texte chiffré. En revanche, pour une clé K donnée, il doit être **unique**.

6.2.1 Pourquoi la réutilisation du nonce est-elle dangereuse ?

Supposons qu'Alice chiffre deux messages différents P et P' avec la même clé K et le même nonce IV .

GCM produira alors le même flot de chiffrement :

$$S = E_K(CTR_i). \quad (28)$$

On aura donc :

$$C_i = P_i \oplus S_i, \quad (29)$$

$$C'_i = P'_i \oplus S_i. \quad (30)$$

Un attaquant peut alors calculer :

$$C_i \oplus C'_i = P_i \oplus P'_i. \quad (31)$$

Le flot de chiffrement s'annule lors du XOR. L'attaquant obtient donc une relation directe entre les deux messages en clair.

La réutilisation du nonce ne compromet donc pas uniquement la confidentialité. Elle peut également compromettre la sécurité du mécanisme d'authentification de GCM.

6.2.2 Bonnes pratiques de gestion

Pour éviter ce problème, plusieurs stratégies sont possibles :

- utiliser un nonce aléatoire suffisamment grand ;
- utiliser un compteur strictement croissant ;
- utiliser une combinaison entre un identifiant de session et un compteur ;
- empêcher toute réinitialisation accidentelle du compteur.

Dans un système distribué, la gestion du nonce doit être conçue avec une attention particulière afin d'éviter qu'un même nonce soit généré simultanément par plusieurs instances utilisant la même clé.

Règle fondamentale de GCM

Pour une clé AES donnée, un nonce ne doit jamais être réutilisé. Cette contrainte est essentielle à la sécurité de GCM.

6.3 Forces et limites du mode GCM

6.3.1 Forces du mode GCM

GCM présente plusieurs avantages importants qui expliquent son adoption dans de nombreux protocoles modernes.

1. Confidentialité et authentification simultanées. GCM fournit dans une même construction :

$$\text{Confidentialité} + \text{Intégrité} + \text{Authenticité}. \quad (32)$$

Il appartient ainsi à la famille des modes AEAD.

2. Exécution parallèle. Le mode CTR permet de calculer indépendamment les blocs de flux de chiffrement :

$$E_K(CTR_1), E_K(CTR_2), \dots, E_K(CTR_n). \quad (33)$$

De même, les opérations internes de GCM peuvent être fortement parallélisées. Cette caractéristique est particulièrement intéressante pour les processeurs modernes.

3. Très bonnes performances. Les processeurs modernes peuvent disposer d'instructions matérielles dédiées à AES et aux opérations nécessaires à GCM. Cela permet d'obtenir des débits élevés avec une faible surcharge.

4. Authentification des données non chiffrées. Grâce aux AAD, GCM peut authentifier des informations qui doivent rester lisibles :

$$A \longrightarrow \text{authenticité mais non chiffré}. \quad (34)$$

Cette propriété est particulièrement utile dans les protocoles réseau.

6.3.2 Limites du mode GCM

Malgré ses avantages, GCM présente certaines contraintes importantes.

1. Sensibilité à la réutilisation du nonce. C'est la principale contrainte opérationnelle de GCM. La réutilisation d'un nonce avec la même clé peut provoquer une compromission grave de la sécurité.

2. Gestion plus complexe des clés et des nonces. Une implémentation correcte doit garantir l'unicité des nonces. Cette exigence devient particulièrement délicate dans les systèmes distribués.

3. Risque associé aux implémentations incorrectes. Une erreur dans la génération du nonce, la gestion des compteurs ou la vérification du tag peut annuler les garanties de sécurité du mode.

4. Coût de l'authentification. Par rapport à un chiffrement seul, GCM effectue des opérations supplémentaires pour calculer le tag. Cette surcharge reste cependant faible sur les architectures disposant d'accélération matérielles.

6.3.3 Synthèse des propriétés

Les principales caractéristiques de GCM peuvent être résumées dans le tableau suivant.

TABLE 6 – Principales propriétés du mode GCM

Propriété	Description
Chiffrement	AES utilisé en mode compteur CTR
Authentification	Fonction GHASH dans $GF(2^{128})$
AAD	Données authentifiées sans être chiffrées
Tag	Permet de détecter les modifications du message
Nonce	Doit être unique pour une clé donnée
Parallélisation	Forte possibilité de parallélisation
Performance	Très élevée avec accélération matérielle
Principal risque	Réutilisation du nonce

6.4 Résistance aux attaques actives

Une attaque active se distingue d'une attaque passive par le fait que l'adversaire tente de modifier, supprimer, réordonner ou injecter des messages dans le système.

GCM est conçu pour détecter ces modifications grâce au tag d'authentification.

6.4.1 Modification du texte chiffré

Supposons qu'un attaquant modifie le texte chiffré :

$$C \longrightarrow C'. \quad (35)$$

Le destinataire recalcule alors :

$$T' = E_K(J_0) \oplus \text{GHASH}_H(A, C'). \quad (36)$$

Comme :

$$C' \neq C, \quad (37)$$

la valeur GHASH obtenue est normalement différente de celle calculée par l'émetteur.

Le destinataire compare alors :

$$T' \stackrel{?}{=} T. \quad (38)$$

Si les deux valeurs diffèrent, le message est rejeté.

Principe de vérification

Un message GCM ne doit être accepté par le destinataire que si le tag d'authentification est correctement vérifié. Le texte chiffré ne doit jamais être utilisé avant cette vérification.

6.4.2 Injection d'un faux message

Un attaquant peut également tenter de fabriquer un nouveau couple (C', T') . Sans connaître la clé secrète K , il doit produire un tag cohérent avec :

$$T' = E_K(J_0) \oplus \text{GHASH}_H(A, C'). \quad (39)$$

La difficulté de cette opération repose notamment sur le caractère secret de la valeur H et sur les propriétés cryptographiques de GHASH.

La probabilité de réussite d'une tentative de forgery dépend notamment de la longueur du tag et du nombre de tentatives effectuées par l'adversaire.

6.4.3 Attaque par modification des AAD

Les données associées sont également protégées contre la modification.

Si :

$$A \longrightarrow A', \quad (40)$$

alors le calcul :

$$\text{GHASH}_H(A', C) \quad (41)$$

produira normalement une valeur différente de :

$$\text{GHASH}_H(A, C). \quad (42)$$

Le tag ne correspondra donc plus et le message sera rejeté.

Cette propriété permet de protéger des champs importants d'un protocole sans avoir à les chiffrer.

6.4.4 Attaque par rejeu

Il convient cependant de distinguer l'intégrité d'un message de la protection contre le rejeu.

GCM permet de détecter la modification d'un message, mais le mode GCM seul ne constitue pas nécessairement un mécanisme complet de protection contre les attaques par rejeu.

Pour empêcher qu'un ancien message authentique soit retransmis, le protocole doit généralement utiliser :

- un numéro de séquence ;
- un compteur ;
- un identifiant unique de message ;
- ou une fenêtre de rejeu.

Ces informations peuvent être placées dans les AAD afin qu'elles soient elles-mêmes authentifiées.

TABLE 7 – GCM face aux principales attaques actives

Attaque	Protection fournie par GCM
Modification du ciphertext	Détectée par la vérification du tag
Modification des AAD	Détectée par GHASH et le tag
Injection d'un faux message	Rendue difficile sans la clé secrète
Réutilisation d'un nonce	Dangereuse ; peut compromettre la sécurité
Rejeu	Nécessite un mécanisme supplémentaire au niveau du protocole

6.4.5 Bilan de la résistance aux attaques actives

Le niveau de protection fourni par GCM peut être résumé ainsi :

En définitive, GCM offre une protection robuste contre la falsification des messages à condition que ses exigences fondamentales soient respectées. La sécurité du mode dépend en particulier de la **non-réutilisation des nonces**, de la protection de la clé secrète et de la vérification systématique du tag avant l'utilisation du message déchiffré.

7 Analyse comparative de CBC et GCM

7.1 Comparaison du niveau de sécurité

7.2 Comparaison face aux attaques actives

7.3 Comparaison des performances

7.4 Avantages et limites de chaque mode

8 Expérimentations et résultats

8.1 Environnement expérimental

8.2 Scénarios de test

8.3 Mesure des performances

8.4 Analyse des résultats

9 Conclusion et perspectives